Guía para construir la base de datos modular con Laravel y PostgreSQL

Informe de implementación basado en esquemas_y_origen_de_tablas.pdf

15 esquemas · 220 tablas de negocio · 12 módulos

28 de octubre de 2026

1 Propósito y resultado esperado

Esta guía explica cómo convertir el modelo del documento en una base PostgreSQL administrada desde Laravel. La arquitectura recomendada es una sola base de datos, una conexión principal y quince esquemas. Los tres compartidos son core, seguridad y comun; los otros doce corresponden a módulos funcionales.

Al terminar se debe poder reconstruir la estructura mediante migraciones, cargar catálogos con seeders y consultar datos mediante modelos Eloquent con relaciones entre esquemas. Este informe es una guía de implementación, no un conjunto completo de 220 migraciones listo para producción. Incluye código completo para crear los quince esquemas, ejemplos reproducibles y un inventario para controlar todas las tablas.

Las migraciones y Schema Builder crean y modifican tablas. Eloquent representa sus registros y relaciones. Crear un modelo con Artisan no crea automáticamente su tabla. PostgreSQL permite claves foráneas entre esquemas de la misma base.

2 Análisis del documento y decisiones previas

El documento reúne entidades que antes estaban duplicadas en los módulos. Persona, estudiante, docente, curso y periodo se mantienen en core. La cuenta de acceso pertenece a seguridad.usuario. Los archivos y empresas se centralizan en comun. Los módulos deben referenciar estas entidades sin crear versiones locales de ellas.

El inventario declara 220 tablas; la suma de las quince secciones coincide. Calidad es una propuesta sin script previo. Gestión de matrículas integra entidades procedentes de Prematrícula y Seguimiento al estudiante. Ambos requieren validación funcional antes de cerrar su estructura física.

No se especifican de manera exhaustiva tipos, longitudes, nulabilidad, valores de estado, acciones de borrado ni todas las restricciones compuestas. Las elecciones de esos puntos en los ejemplos son propuestas técnicas y deben incorporarse a un diccionario aprobado. No se puede certificar la normalización de todas las tablas únicamente con este inventario.

Dos relaciones necesitan confirmación explícita: seguimiento_estudiante.atencion_derivacion.id_caso_bienestar probablemente referencia bienestar.caso_bienestar.id_caso; gestion_matriculas.matricula.id_solicitud_prematricula probablemente referencia prematricula.solicitud_prematricula.id_solicitud. El documento muestra esas columnas sin flecha de FK. Añadirlas solo después de validar el significado y su nulabilidad.

Calidad debe consumir vistas publicadas por los otros módulos, según el documento. Esas vistas no están definidas: acordar nombre, columnas, filtros y permisos; crearlas después de las tablas que consultan.

3 Reglas comunes para todos los equipos

Conservar nombres en español, singular, minúsculas y snake_case. Usar siempre esquema.tabla en migraciones, modelos, pivotes y consultas. Mantener las PK originales id_persona, id_estudiante y similares. Propuesta de tipo común: bigserial mediante bigIncrements; FK compatibles mediante bigInteger. PostgreSQL no tiene enteros unsigned.

Usar varchar para documentos, RUC, códigos y teléfonos, conservando ceros iniciales. Usar decimal para notas, créditos fraccionarios, montos y porcentajes; evitar float para valores que requieren exactitud. Usar date para fechas civiles y timestampTz para eventos; acordar UTC para almacenamiento y America/Lima para presentación. Usar jsonb para parámetros y datos de auditoría cuando su estructura sea variable.

Un campo opcional lleva nullable por una razón de negocio. No aplicar nullable a todo para que la migración pase. Toda FK debe tener destino, tipo compatible, política de borrado y, cuando se consulte por ella, índice. PostgreSQL no crea automáticamente un índice en la columna hija de una FK.

Dar nombres cortos y explícitos a índices y restricciones. PostgreSQL limita por defecto los identificadores a 63 bytes; nombres generados con esquemas y columnas largas pueden truncarse y colisionar. Ejemplo: pre_det_sol_curso_uq.

No agregar timestamps a todas las tablas sin revisar el documento. Las tablas de negocio usan creado_en, registrada_en o revisado_en; configurar Eloquent según cada tabla. Las tablas adicionales de Laravel no se cuentan dentro de las 220.

4 Preparar el entorno y crear el proyecto

Referencia de los ejemplos: Laravel 13.x y PostgreSQL 18. Antes de instalar, revisar los requisitos de PHP y extensiones de esa versión en la documentación oficial. Instalar Composer y habilitar pdo_pgsql. pgAdmin es opcional; psql sirve para las tareas administrativas. Fijar versiones con composer.lock para todos los equipos.

```
php -v
composer --version
php -m
composer create-project laravel/laravel portal-escuela "^13.0"
cd portal-escuela
php artisan --version
```

En php -m debe aparecer pdo_pgsql. Si no aparece, habilitar la extensión en el php.ini que usa la consola y reiniciar el servidor PHP. Si Artisan usa SQLite al crear el proyecto, configurar PostgreSQL antes de ejecutar las migraciones del sistema.

5 Crear la base PostgreSQL

Conectarse como administrador mediante psql o pgAdmin. Los siguientes comandos son de desarrollo local. Sustituir la contraseña; no incluir credenciales reales en Git. CREATE DATABASE se ejecuta fuera de una transacción y antes de las migraciones de Laravel.

```
CREATE ROLE portal_owner LOGIN PASSWORD 'CAMBIAR_CLAVE_LOCAL';
CREATE DATABASE portal_escuela
  OWNER portal_owner ENCODING 'UTF8' TEMPLATE template0;
```

Abrir una conexión a portal_escuela como administrador y preparar public para las tablas técnicas y el historial de migraciones.

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE, CREATE ON SCHEMA public TO portal_owner;
```

En desarrollo, portal_owner ejecuta las migraciones y la aplicación. En producción separar el usuario propietario que despliega del usuario que atiende solicitudes. El segundo recibe CONNECT, USAGE de esquemas y permisos de datos concretos; no CREATE, DROP ni propiedad de las tablas.

6 Configurar Laravel para PostgreSQL

Editar .env con la base ya creada. Este ejemplo usa sesiones y caché en archivos para no depender de tablas técnicas que todavía no se han migrado.

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=portal_escuela
DB_USERNAME=portal_owner
DB_PASSWORD=CAMBIAR_CLAVE_LOCAL
SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync
```

En config/database.php, dentro de connections.pgsql, conservar el resto de la configuración del proyecto y fijar search_path a public. Evitar colocar los quince esquemas en search_path: las tablas de negocio se referencian por nombre completo.

```
'search_path' => 'public',

php artisan config:clear
php artisan tinker
```

Dentro de Tinker verificar la conexión:

```
DB::selectOne('select current_database() as bd');
DB::selectOne('show search_path');
```

Revisar las migraciones iniciales de Laravel antes del primer migrate. No crear public.users si se usará seguridad.usuario. Retirar esa creación del proyecto nuevo y decidir por separado si se necesitan sessions, jobs o password_reset_tokens. No eliminar migraciones que ya se ejecutaron en un entorno compartido; crear migraciones correctivas.

7 Organizar migraciones en fases

Para el primer despliegue, organizar por fases globales: 1 crear esquemas; 2 crear todas las tablas y columnas con PK y restricciones únicas; 3 añadir todas las FK; 4 añadir checks y vistas; 5 cargar catálogos. Así se resuelven referencias cruzadas sin desactivar integridad referencial.

Mantener inicialmente todos los archivos en database/migrations. Laravel ordena por el nombre del archivo, por lo que los prefijos de fecha y hora deben reflejar las fases. El nombre de un directorio no controla por sí solo el orden. Si se eligen subdirectorios, registrar rutas y probar el orden global; no asumir que migrate los recorre automáticamente.

```
2026_10_28_000001_create_business_schemas.php
2026_10_28_010001_create_core_pais.php
... todas las tablas de los quince esquemas ...
2026_10_28_080001_add_core_foreign_keys.php
... claves foráneas por esquema ...
2026_10_28_090001_create_published_views.php
```

Los puntos suspensivos representan archivos por implementar; no forman parte de nombres reales. Acordar prefijos únicos entre equipos y no cambiar el nombre de una migración ejecutada. En ampliaciones futuras, cada cambio nuevo crea su propia migración posterior.

8 Crear los quince esquemas desde una migración

Ejecutar php artisan make:migration create_business_schemas y ubicar este archivo antes de todas las migraciones de negocio. Reemplazar su contenido por el siguiente. DB::statement usa SQL de PostgreSQL desde Laravel para la operación de esquema; las tablas se crean con Schema Builder.

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Support\Facades\DB;

return new class extends Migration {
    private array $schemas = [
        'core', 'seguridad', 'comun', 'prematricula',
        'bienestar', 'investigacion_formativa',
        'seguimiento_estudiante', 'web_institucional',
        'calidad', 'gestion_horarios', 'cotecu',
        'gestion_practicas', 'tesis',
        'gestion_matriculas', 'seguimiento_egresado',
    ];

    public function up(): void
    {
        foreach ($this->schemas as $schema) {
            DB::statement('CREATE SCHEMA "'.$schema.'"');
        }
    }

    public function down(): void
    {
        foreach (array_reverse($this->schemas) as $schema) {
            DB::statement('DROP SCHEMA "'.$schema.'"');
        }
    }
};
```

La lista es fija, nunca proviene de un formulario. No usar IF NOT EXISTS para ocultar un estado inesperado en una instalación nueva. down no usa CASCADE: solo puede borrar esquemas vacíos después de revertir vistas, FK y tablas. Si hay objetos externos, detener y revisar.

9 Patrón completo de tabla y clave foránea

Ejemplo pequeño y fiel a las columnas del inventario: core.pais. Las longitudes son propuestas. Crear el archivo con php artisan make:migration create_core_pais.

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('core.pais', function (Blueprint $t) {
            $t->bigIncrements('id_pais');
            $t->string('nombre', 120);
            $t->unique('nombre', 'core_pais_nombre_uq');
        });
    }
    public function down(): void
    {
        Schema::dropIfExists('core.pais');
    }
};
```

En la migración que crea core.persona declarar id_pais_nacionalidad como bigInteger; en la fase de FK añadir la relación siguiente. Es un fragmento dentro de up de una migración que importa Schema y Blueprint. No ejecutar antes de que ambas tablas existan.

```
Schema::table('core.persona', function (Blueprint $t) {
    $t->foreign('id_pais_nacionalidad', 'persona_pais_fk')
      ->references('id_pais')->on('core.pais')
      ->restrictOnDelete();
});
```

En down de esa migración: Schema::table('core.persona', fn (Blueprint $t) => $t->dropForeign('persona_pais_fk')). La nulabilidad de la columna y su índice se definen en la migración de creación. No usar constrained() sin parámetros en este modelo: Laravel inferiría nombres de tablas y PK que no coinciden con el inventario.

10 Ejemplo completo del módulo de prematrícula

Estas tres tablas ilustran cómo desarrollar un módulo entero. Se conservan las columnas del PDF; los tipos, longitudes, nulabilidad y dos restricciones adicionales son propuestas. Se exige una solicitud por estudiante y periodo y un curso por solicitud. Validar estas dos reglas si el proceso permite reenvíos como solicitudes nuevas.

Crear una migración create_prematricula_tables posterior a la de esquemas. Importa Migration, Blueprint y Schema como el ejemplo anterior; usar los métodos siguientes dentro de la clase anónima.

```
public function up(): void
{
    Schema::create('prematricula.cronograma_prematricula',
        function (Blueprint $t) {
            $t->bigIncrements('id_cronograma');
            $t->bigInteger('id_periodo');
            $t->timestampTz('fecha_inicio');
            $t->timestampTz('fecha_fin');
            $t->string('estado', 30);
            $t->unique('id_periodo', 'pre_crono_periodo_uq');
        });
    Schema::create('prematricula.solicitud_prematricula',
        function (Blueprint $t) {
            $t->bigIncrements('id_solicitud');
            $t->bigInteger('id_estudiante');
            $t->bigInteger('id_periodo');
            $t->string('estado', 30);
            $t->text('observacion')->nullable();
            $t->timestampTz('enviada_en')->nullable();
            $t->bigInteger('revisado_por')->nullable();
            $t->timestampTz('revisado_en')->nullable();
            $t->unique(['id_estudiante', 'id_periodo'],
                'pre_sol_est_periodo_uq');
            $t->index('id_periodo', 'pre_sol_periodo_idx');
            $t->index('revisado_por', 'pre_sol_revisor_idx');
        });
    Schema::create('prematricula.detalle_solicitud',
        function (Blueprint $t) {
            $t->bigIncrements('id_detalle');
            $t->bigInteger('id_solicitud');
            $t->bigInteger('id_curso_plan');
            $t->bigInteger('id_seccion_preferida')->nullable();
            $t->string('estado', 30);
            $t->text('motivo_rechazo')->nullable();
            $t->unique(['id_solicitud', 'id_curso_plan'],
                'pre_det_sol_curso_uq');
            $t->index('id_curso_plan', 'pre_det_curso_idx');
            $t->index('id_seccion_preferida', 'pre_det_sec_idx');
        });
}
public function down(): void
{
    Schema::dropIfExists('prematricula.detalle_solicitud');
    Schema::dropIfExists('prematricula.solicitud_prematricula');
    Schema::dropIfExists('prematricula.cronograma_prematricula');
}
```

Crear otra migración posterior a TODAS las tablas para las siete FK del módulo. El destino revisado_por es seguridad.usuario; la sección preferida pertenece a core, no a gestion_horarios.

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    private function links(): array
    {
        return [
          ['cronograma_prematricula', 'id_periodo',
           'core.periodo_academico', 'id_periodo', 'pre_cr_per_fk'],
          ['solicitud_prematricula', 'id_estudiante',
           'core.estudiante', 'id_estudiante', 'pre_so_est_fk'],
          ['solicitud_prematricula', 'id_periodo',
           'core.periodo_academico', 'id_periodo', 'pre_so_per_fk'],
          ['solicitud_prematricula', 'revisado_por',
           'seguridad.usuario', 'id_usuario', 'pre_so_rev_fk'],
          ['detalle_solicitud', 'id_solicitud',
           'prematricula.solicitud_prematricula',
           'id_solicitud', 'pre_de_sol_fk'],
          ['detalle_solicitud', 'id_curso_plan',
           'core.curso_plan', 'id_curso_plan', 'pre_de_cur_fk'],
          ['detalle_solicitud', 'id_seccion_preferida',
           'core.seccion', 'id_seccion', 'pre_de_sec_fk'],
        ];
    }
    public function up(): void
    {
        foreach ($this->links() as [$table,$col,$to,$pk,$name]) {
            Schema::table('prematricula.'.$table,
                function (Blueprint $t) use ($col,$to,$pk,$name) {
                    $t->foreign($col, $name)
                      ->references($pk)->on($to)->restrictOnDelete();
                });
        }
    }
    public function down(): void
    {
        foreach (array_reverse($this->links()) as $link) {
            Schema::table('prematricula.'.$link[0],
                fn (Blueprint $t) => $t->dropForeign($link[4]));
        }
    }
};
```

Añadir después un CHECK fecha_fin > fecha_inicio al cronograma mediante DB::statement y eliminarlo en down. La sección preferida debe corresponder al curso solicitado y al periodo de la solicitud: las FK individuales no garantizan esa coherencia. Validarla en el servicio y, si se exige protección ante escrituras externas, diseñar restricciones compuestas o un trigger revisado.

11 Modelos Eloquent con esquema y PK personalizada

Crear modelos sin generar migraciones duplicadas. Artisan organiza las clases en subdirectorios; el nombre de clase no cambia el nombre físico de la tabla cuando se declara $table.

```
php artisan make:model Core/Pais
php artisan make:model Core/Persona
php artisan make:model Core/Estudiante
php artisan make:model Prematricula/SolicitudPrematricula
php artisan make:model Prematricula/DetalleSolicitud
```

Modelo mínimo completo de Pais. No tiene timestamps porque la tabla del PDF no los tiene.

```php
<?php
namespace App\Models\Core;
use Illuminate\Database\Eloquent\Model;

class Pais extends Model
{
    protected $table = 'core.pais';
    protected $primaryKey = 'id_pais';
    public $timestamps = false;
    protected $fillable = ['nombre'];
}
```

Modelo de solicitud. Mantiene las fechas de negocio mediante casts; no inventa created_at ni updated_at. Se muestran dos relaciones completas y sus claves explícitas.

```php
<?php
namespace App\Models\Prematricula;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use App\Models\Core\Estudiante;

class SolicitudPrematricula extends Model
{
    protected $table = 'prematricula.solicitud_prematricula';
    protected $primaryKey = 'id_solicitud';
    public $timestamps = false;
    protected $fillable = [
        'id_estudiante', 'id_periodo', 'estado', 'observacion',
        'enviada_en', 'revisado_por', 'revisado_en',
    ];
    protected function casts(): array
    {
        return ['enviada_en' => 'datetime',
                'revisado_en' => 'datetime'];
    }
    public function estudiante(): BelongsTo
    {
        return $this->belongsTo(Estudiante::class,
            'id_estudiante', 'id_estudiante');
    }
    public function detalles(): HasMany
    {
        return $this->hasMany(DetalleSolicitud::class,
            'id_solicitud', 'id_solicitud');
    }
}
```

Completar Estudiante y DetalleSolicitud con sus tablas, PK y fillable antes de ejecutar las relaciones. Para core.persona, si se quiere que Eloquent escriba creado_en automáticamente, usar const CREATED_AT = 'creado_en'; const UPDATED_AT = null; y mantener $timestamps = true. Elegir esa estrategia por tabla; no combinarla con $timestamps = false esperando una escritura automática.

Los campos fillable solo deben recibir valores preparados por el servicio. No pasar request()->all() al modelo; id_estudiante, revisado_por y estado deben derivarse de la identidad y las reglas autorizadas del servidor.

12 Resolver tablas puente y claves compuestas

El documento contiene tablas como core.docente_linea, seguridad.rol_permiso, bienestar.estudiante_deporte y gestion_matriculas.nota. Eloquent no admite PK compuestas en modelos ordinarios. Decidir por tabla entre las siguientes estrategias.

Para una asociación simple N:M conservar PK compuesta y usar belongsToMany o Query Builder con ambas claves. No usar find, save o destroy de un modelo que presuponga una sola PK. Ejemplo de migración de docente_linea:

```
Schema::create('core.docente_linea', function (Blueprint $t) {
    $t->bigInteger('id_docente');
    $t->bigInteger('id_linea');
    $t->primary(['id_docente', 'id_linea'], 'doc_linea_pk');
    $t->index('id_linea', 'doc_linea_linea_idx');
});
```

Añadir ambas FK en la fase de relaciones. En el modelo Docente, el siguiente método requiere que LineaInvestigacion esté configurado con core.linea_investigacion e id_linea.

```
public function lineas()
{
    return $this->belongsToMany(
        LineaInvestigacion::class, 'core.docente_linea',
        'id_docente', 'id_linea', 'id_docente', 'id_linea'
    );
}
```

Para una entidad que necesita CRUD propio con Eloquent, propuesta alternativa: añadir una PK técnica id y conservar unique sobre la combinación original. Por ejemplo, nota tendría id más unique(id_detalle, id_componente). Es un cambio al modelo del PDF: documentarlo y aprobarlo; no eliminar la unicidad original. Las tablas de clave simple no autoincremental, como comun.dia_no_laborable con fecha, requieren $incrementing = false y $keyType = 'string'.

Tablas como satisfaccion_egresado o perfil_docente_web pueden usar su FK como PK simple si así se confirma en el PDF. No agregar automáticamente otra PK por el solo hecho de que la primera columna sea una FK.

13 Autenticación central y acceso por módulo

Usar seguridad.usuario como única cuenta de acceso. La persona existe en core.persona y puede tener perfil de estudiante o docente sin duplicar sus datos personales. Los roles de la aplicación no son roles de PostgreSQL.

Configurar el proveedor users de config/auth.php para que su model sea App\Models\Seguridad\Usuario::class. La tabla debe existir antes de probar el login. Modelo orientativo con el hash personalizado:

```
<?php
namespace App\Models\Seguridad;
use Illuminate\Foundation\Auth\User as Authenticatable;

class Usuario extends Authenticatable
{
    protected $table = 'seguridad.usuario';
    protected $primaryKey = 'id_usuario';
    const CREATED_AT = 'creado_en';
    const UPDATED_AT = null;
    protected $hidden = ['contrasena_hash', 'remember_token'];
    public function getAuthPasswordName(): string
    {
        return 'contrasena_hash';
    }
}
```

Crear contraseñas con Hash::make y nunca guardar texto plano. La columna remember_token debe ser nullable si se habilita recordar sesión. Ejemplo dentro del controlador de login que ya valida correo y contraseña:

```
$ok = Auth::attempt([
    'correo_institucional' => $correo,
    'password' => $clave,
    'estado' => 'activo',
]);
if ($ok) {
    $request->session()->regenerate();
}
```

El valor activo es una propuesta de catálogo y debe coincidir con la BD. Al autorizar rutas consultar usuario_rol, rol, rol_permiso, permiso y modulo_sistema. Considerar estado, fecha_inicio y fecha_fin de la asignación, así como módulo y permiso solicitado. Aplicar middleware o Policies en cada endpoint; mostrar u ocultar un botón no protege el acceso.

La recuperación de contraseña de Laravel no usa automáticamente seguridad.token_usuario. Implementar un repositorio compatible o conservar una tabla técnica del mecanismo estándar, según la decisión del equipo. No habilitar verificación de correo sin adaptar también sus columnas y contratos. No asumir que instalar un paquete de permisos lo conecta con estas tablas personalizadas.

14 Transacciones validación y carga inicial

Usar transacciones para operaciones que cruzan tablas o esquemas en la misma conexión. En una solicitud de prematrícula, validar estudiante autenticado, ventana del cronograma, periodo, cursos habilitados, requisitos y duplicados. Después crear cabecera y detalles en una sola transacción.

```
use Illuminate\Support\Facades\DB;
use App\Models\Prematricula\SolicitudPrematricula;

$solicitud = DB::transaction(function () use ($datos, $items) {
    $solicitud = SolicitudPrematricula::create($datos);
    $solicitud->detalles()->createMany($items);
    return $solicitud;
});
```

Este fragmento supone modelos completos y datos previamente autorizados; no es un endpoint listo para publicar. Las restricciones unique siguen siendo necesarias ante solicitudes concurrentes. Si se controla capacidad o saldo, bloquear las filas relevantes dentro de la transacción y volver a verificar el valor.

Crear seeders por esquema. Cargar primero catálogos y estructura de core, después módulos y permisos de seguridad, luego catálogos de comun y de cada módulo. Cargar docentes, estudiantes y demás datos de prueba solo cuando existan sus referencias. Usar datos ficticios para las pruebas.

```
php artisan make:seeder CoreCatalogosSeeder
php artisan make:seeder SeguridadCatalogosSeeder
```

En run de CoreCatalogosSeeder, un ejemplo idempotente sin fijar IDs:

```
DB::table('core.pais')->updateOrInsert(
    ['nombre' => 'Perú'], ['nombre' => 'Perú']
);
```

Llamar los seeders desde DatabaseSeeder en orden explícito. Cargar seguridad.modulo_sistema con los doce nombres de esquema de módulos del inventario, sin core, seguridad ni comun. No sembrar usuarios reales ni contraseñas públicas por defecto.

15 Ruta de implementación por esquema

La siguiente ruta indica qué implementar y qué revisar en cada esquema. Las FK se agregan cuando todas las tablas ya existen. El anexo final contiene las 220 tablas para controlar que ninguna se omita.

core

Crear primero país y ubigeo, facultad, periodos, modalidades, áreas, cursos y ambientes. Después persona, escuela y departamento, plan y curso_plan, perfiles de docente y estudiante, líneas y secciones. Incorporar tablas de prerrequisitos, equivalencias y docente_linea. Mantener este esquema sin dependencia hacia los módulos. Validar pertenencia del plan a la escuela y evitar prerrequisitos del mismo curso o ciclos entre requisitos; una FK sola no evita esos ciclos.

seguridad

Crear modulo_sistema y usuario; después rol y permiso, sus asignaciones, solicitud_acceso, tokens y registros. La dependencia externa central es core.persona. Definir unicidad de correo y si se compara sin distinguir mayúsculas. La auditoría no se vuelve inalterable por el nombre de la tabla: restringir UPDATE y DELETE del usuario de ejecución y definir captura transaccional o triggers para los cambios que se deben registrar.

comun

Crear sector_economico antes de empresa y contacto_empresa. Archivo y notificacion referencian seguridad; empresa y contactos también core. parametro_sistema puede necesitar unique(id_modulo, clave), regla propuesta. dia_no_laborable usa fecha como clave natural. Guardar el archivo físico en almacenamiento y sus metadatos en comun.archivo; controlar permisos de descarga fuera de la URL pública.

prematricula

Aplicar el ejemplo de las secciones 10 y 11. Cronograma depende del periodo; solicitud depende de estudiante, periodo y usuario revisor; detalle depende de solicitud, curso_plan y sección preferida. Probar que una solicitud incompleta no deje cabecera huérfana. Confirmar reglas sobre reenvíos, cambios de periodo y cierre del cronograma.

bienestar

Comenzar por especialidad, tipo_caso, tipo_condicion, deporte y beneficio; continuar con especialistas, perfil socioeconómico, solicitudes y casos, horarios, citas, derivaciones y seguimiento. Cargar evidencias mediante comun.archivo. Restringir consultas de datos sensibles por función y por caso asignado. Validar fin mayor que inicio y decidir cómo impedir citas simultáneas del especialista: validación con bloqueo o restricción de exclusión de PostgreSQL, según la granularidad acordada.

investigacion_formativa

Crear tipo_producto, grupo, rubrica y criterio; luego proyecto, entregable, entrega, evaluacion y sus detalles. Usar secciones, docentes, estudiantes y líneas de core. Validar que proyecto, grupo y entregable pertenezcan a la misma sección y que cada criterio evaluado corresponda a la rúbrica del entregable. La FK por separado no verifica estas combinaciones.

seguimiento_estudiante

Crear catálogos de brechas, SAT y atención; después perfiles, situación académica, alertas y tutorías. Implementar atención y sus motivos, compromisos, derivaciones y evidencias; después el bloque de nivelación. Confirmar la FK opcional hacia bienestar. Definir quién calcula situación_academica y cómo evita inconsistencias con las notas oficiales de gestion_matriculas.

web_institucional

Crear categorías, publicaciones y archivos adjuntos; campañas y modalidades de admisión; documentos oficiales y perfiles públicos. plan_estudios_publicacion y perfil_docente_web extienden datos de core; no duplicar planes ni docentes. Implementar slug único y filtrar explícitamente el estado de publicación. Separar las rutas públicas de las de aprobación de contenido.

calidad

Crear proceso, indicadores, metas, mediciones, estándares, evidencias y planes de mejora según su inventario. proceso tiene una autorrelación al proceso padre; definir raíz con padre nullable y prevenir ciclos jerárquicos en el servicio. No asumir aprobación del diseño por estar listado: validar el módulo con sus responsables. Consumir las vistas publicadas por otros módulos y documentar la periodicidad de las mediciones.

gestion_horarios

Crear oferta_curso y actividades, asignaciones docentes, bloques, disponibilidad y bloqueos; luego horario, detalle, restricciones y revisiones. core.seccion sigue siendo la oferta académica compartida. Un horario no debe duplicar docentes, ambientes o secciones para un mismo periodo y bloque vigente. Una unicidad local al detalle puede ser insuficiente si hay varias versiones activas; definir cómo se publica una versión oficial.

cotecu

Crear miembros, listas e ítems de cotejo; después sílabos, revisiones de sílabos y curriculares, detalles e informes técnicos. Usar core.curso_plan, periodo y docente como referencias. Conservar versiones de documentos y listas. Validar que cada ítem respondido pertenezca a la lista utilizada en la revisión.

gestion_practicas

Crear convocatoria y sus observaciones, practica, postulacion, seguimiento, documentos y sus observaciones, criterios y evaluación de acuerdo con el inventario. Las empresas son comun.empresa y los estudiantes y supervisores son core. Los documentos se vinculan a comun.archivo. Validar fechas de convocatoria, periodo y horas de práctica y trazabilidad de la aprobación antes de permitir el cierre.

tesis

Crear catálogos y proyecto_tesis; después estudiantes participantes, solicitudes, resoluciones, asesores y jurados, cronograma y presupuesto. Continuar con entregas, rúbricas, revisiones y observaciones, historial y sustentación, resultados y actas. Todos los participantes deben referenciar las identidades de core. Confirmar las reglas de composición del jurado, número de autores, etapas y versiones con el responsable académico.

gestion_matriculas

Crear cronograma, límites y matrícula; continuar con detalles, componentes de evaluación, notas y asistencia; terminar con actas y reservas. Confirmar relación hacia prematricula. Un componente de nota debe pertenecer a la sección del detalle matriculado. nota y asistencia requieren revisar PK compuestas; en asistencia la fecha forma parte del identificador propuesto por el modelo. Definir cierre de actas y quién puede corregir notas cerradas.

seguimiento_egresado

Comenzar con sus doce catálogos, cohortes, competencias y objetivos. Continuar con egresado y su historial, grados, contactos y formación; importación de padrones; campañas e invitaciones; mediciones y respuestas; indicadores, reportes y mejoras. La cuenta de usuario no sustituye el expediente del egresado. Tokens de invitación se guardan como hash y deben tener un ciclo de vida definido; la expiración no aparece en todas las tablas del inventario y requiere una decisión adicional.

16 Ejecutar y comprobar la instalación

Antes de migrar, confirmar que todas las columnas, PK y FK estén implementadas, no solo las tablas del ejemplo. Ejecutar sobre una base nueva de desarrollo.

```
php artisan migrate:status
php artisan migrate --pretend
php artisan migrate
php artisan db:seed
php artisan migrate:status
```

--pretend ayuda a revisar el SQL; no demuestra que PostgreSQL aceptará las restricciones ni que el modelo sea correcto. El resultado esperado de la última instrucción es que todas las migraciones estén ejecutadas. La tabla public.migrations y las tablas técnicas acordadas son adicionales a las 220 de negocio.

Consulta para comparar las cantidades por esquema. Debe devolver quince filas y todas con resultado OK; los esquemas sin tablas se muestran como cero.

```
WITH esperado(esquema, cantidad) AS (
 VALUES ('core',22), ('seguridad',10), ('comun',7),
 ('prematricula',3), ('bienestar',18),
 ('investigacion_formativa',11), ('seguimiento_estudiante',23),
 ('web_institucional',13), ('calidad',12),
 ('gestion_horarios',11), ('cotecu',9),
 ('gestion_practicas',10), ('tesis',22),
 ('gestion_matriculas',9), ('seguimiento_egresado',40)
)
SELECT e.esquema, e.cantidad AS esperado,
 count(t.table_name) AS encontrado,
 CASE WHEN count(t.table_name)=e.cantidad
      THEN 'OK' ELSE 'REVISAR' END AS resultado
FROM esperado e
LEFT JOIN information_schema.tables t
 ON t.table_schema=e.esquema AND t.table_type='BASE TABLE'
GROUP BY e.esquema,e.cantidad
ORDER BY e.esquema;
```

La cantidad no prueba que las tablas correctas estén presentes: comparar además sus nombres con el anexo. Ejecutar con el propietario para evitar que los permisos oculten objetos. Para revisar claves foráneas:

```
SELECT conrelid::regclass AS tabla, conname AS restriccion,
       confrelid::regclass AS destino,
       pg_get_constraintdef(oid) AS definicion
FROM pg_constraint
WHERE contype='f'
ORDER BY conrelid::regclass::text, conname;
```

Agregar pruebas de integración sobre PostgreSQL, no solo SQLite, porque se usan esquemas, tipos y restricciones específicas. Probar una FK inválida, un duplicado, una operación válida entre esquemas, rollback de transacción, persistencia de fechas personalizadas y acceso denegado por permiso o rol vencido. Probar dos solicitudes concurrentes cuando exista cupo o unicidad de negocio.

En una base desechable probar también el ciclo de reversión completo y nueva migración. Las FK se revierten primero; después las tablas y finalmente los esquemas. No usar migrate:fresh ni migrate:reset sobre una base compartida o con información que deba conservarse.

17 Desarrollo paralelo entre cuatro personas

Los doce módulos pueden funcionar como sistemas diferenciados y mantener una sola base PostgreSQL. La división de trabajo se hará por propiedad de esquemas, con una integración central. Cada persona desarrolla localmente contra una copia completa de la estructura; no necesita esperar a que los otros sistemas tengan pantallas, controladores o lógica terminada.

La propuesta reparte las 220 tablas en cuatro bloques de 53, 50, 61 y 56 tablas. El número de tablas es una referencia de carga, no una estimación de horas: seguridad, horarios, notas y las reglas de tesis pueden requerir más esfuerzo por tabla. La persona 1 asume además la coordinación de integración.

Asignación de responsabilidad

Persona 1 y coordinación: core 22, seguridad 10, comun 7, prematricula 3 y gestion_horarios 11. Total 53 tablas. Implementa las entidades compartidas, la identidad central, la oferta académica y la demanda previa de cursos. Mantiene la conexión, la migración inicial de los quince esquemas y el historial central.

Persona 2 y trayectoria del estudiante: bienestar 18, seguimiento_estudiante 23 y gestion_matriculas 9. Total 50 tablas. Implementa atención, tutorías, nivelación, matrícula, notas y asistencia. Es responsable de resolver la relación entre derivaciones estudiantiles y casos de bienestar, y de coordinar con la persona 1 la referencia a prematrícula.

Persona 3 y vinculación e investigación formativa: seguimiento_egresado 40, gestion_practicas 10 e investigacion_formativa 11. Total 61 tablas. Implementa expedientes de egresados, encuestas e indicadores laborales, postulaciones y prácticas, así como proyectos formativos por sección. Tiene más tablas, pero muchas son catálogos o asociaciones; revisar su carga al finalizar el primer bloque.

Persona 4 y gestión académica institucional: tesis 22, cotecu 9, web_institucional 13 y calidad 12. Total 56 tablas. Implementa proyectos y sustentaciones, revisión curricular, publicación institucional e indicadores de calidad. Coordina las vistas que Calidad consume y valida el carácter propuesto de su modelo.

Cada persona es dueña de las migraciones, modelos, relaciones, seeders y pruebas de sus esquemas. La lista exacta de tablas de cada uno está en el anexo de este informe. Ningún equipo crea una persona, cuenta, empresa, curso o estudiante alternativo dentro de su módulo.

Fase inicial y contrato común

Antes de crear tablas, realizar una revisión conjunta del diccionario. Para cada columna acordar tipo y longitud, PK o FK, nullable, default y regla de validación. Para cada FK registrar destino completo, columnas, política de borrado y nombre corto. Marcar por separado las reglas propuestas que aún requieren validación funcional.

La persona 1 publica primero el contrato mínimo de core, seguridad y comun: nombres, columnas, PK y tipos. Las personas 2, 3 y 4 revisan si esos contratos cubren sus referencias. Una vez acordados, los cuatro pueden escribir sus migraciones en paralelo. No necesitan esperar a que los 39 modelos compartidos tengan todos sus métodos implementados.

La persona 1 también crea una sola migración de los quince esquemas y configura PostgreSQL. Esta es una tarea de arranque breve; no es necesario que termine todos sus cinco esquemas antes de que los demás comiencen. Los equipos pueden avanzar con migraciones, modelos locales, catálogos y reglas mientras se integra la estructura compartida.

Fase de tablas en paralelo

Las cuatro personas crean las tablas y columnas de sus bloques con PK, restricciones únicas e índices, todavía sin FK. No usar modelos Eloquent dentro de migraciones para crear estructura ni para depender de la versión actual del código del modelo. Usar Schema Builder y SQL de PostgreSQL cuando haga falta.

Los modelos pueden desarrollarse desde esta fase: declarar namespace, tabla con esquema, PK y fechas. Las relaciones internas se pueden implementar inmediatamente. Si una clase externa todavía no existe, acordar su namespace y completar la integración antes de ejecutar consultas que la instancian; un nombre acordado no es una clase funcional.

Integrar diariamente las migraciones de tablas. La primera meta conjunta es una base local con quince esquemas y 220 tablas, aunque todavía falten FK y datos. Esta meta permite trabajar en paralelo sin simular tablas compartidas distintas del modelo definitivo.

Fase de claves foráneas y coherencia

Después de integrar todas las tablas, cada persona agrega las FK que NACEN en sus propias tablas, incluso cuando apuntan al esquema de otra persona. Así hay un único responsable por restricción y se evitan implementaciones duplicadas. La persona dueña del destino revisa compatibilidad y política de borrado.

Ejemplo: la persona 2 crea la FK de gestion_matriculas.matricula hacia prematricula.solicitud_prematricula cuando se confirme esa relación; la persona 1 revisa el destino. La persona 3 crea las FK de prácticas hacia comun.empresa y core.estudiante; la persona 1 no las agrega desde los esquemas compartidos.

En esta fase comprobar que cada relación sea válida y que las reglas entre varias columnas sean consistentes. Una entrega evaluada debe usar la rúbrica correcta; una nota debe corresponder a la sección matriculada. Resolver estos controles mediante restricciones o servicios transaccionales, según el caso, y registrar las decisiones.

Fase de datos iniciales y pruebas

Cada persona entrega seeders de catálogos de su bloque. La persona 1 mantiene DatabaseSeeder y coordina el orden global: catálogos y entidades de core, módulos y permisos de seguridad, catálogos de comun, catálogos de módulos y datos de prueba dependientes. Un seeder no debe depender de IDs inventados en otro entorno; consultar las claves de negocio acordadas.

Cada persona ejecuta sus pruebas contra PostgreSQL local con la estructura integrada. Los cuatro prueban un flujo completo que involucre al menos una referencia externa de su bloque. La aceptación exige que una FK inválida falle, un duplicado se rechace y una transacción fallida no deje datos parciales.

Un repositorio de base de datos y cuatro ramas

Mantener un historial único de migraciones. Si los doce sistemas se desarrollan en repositorios distintos, designar un proyecto Laravel central como dueño de las migraciones y de su ejecución. Los sistemas consumen el mismo contrato; no deben ejecutar doce copias independientes del mismo historial contra la base compartida.

Para este proyecto central usar cuatro ramas: bd/persona1-compartidos, bd/persona2-estudiantes, bd/persona3-vinculacion y bd/persona4-institucional. Cada persona trabaja en su propia base local con credenciales propias. Integrar cambios mediante revisión, descargar el código integrado y ejecutar migrate localmente. No programar todos directamente sobre una sola base de pruebas sin control.

Guardar todas las migraciones inicialmente en database/migrations. Identificar al responsable mediante el nombre del archivo y reservar rangos de numeración; no separar carpetas sin configurar su carga. Ejemplo de convención inicial:

```
2026_10_28_000001_create_business_schemas.php
2026_10_28_010101_p1_create_core_pais.php
2026_10_28_010201_p2_create_bienestar_especialidad.php
2026_10_28_010301_p3_create_egresado_cat_area.php
2026_10_28_010401_p4_create_tesis_tipo_solicitud.php
2026_10_28_080101_p1_add_core_foreign_keys.php
2026_10_28_080201_p2_add_bienestar_foreign_keys.php
2026_10_28_080301_p3_add_practicas_foreign_keys.php
2026_10_28_080401_p4_add_tesis_foreign_keys.php
2026_10_28_090401_p4_create_quality_source_views.php
```

Los nombres son ejemplos de la convención; cada bloque requiere más archivos. Reservar 010100 a 010199 para las tablas de la persona 1; 010200 a 010299 para la 2; 010300 a 010399 para la 3; 010400 a 010499 para la 4. Aplicar rangos análogos de 080100 a 080499 para las FK. Las 220 tablas caben en estos rangos si se usa una migración por tabla. Los prefijos completos son identificadores de orden, no horas reales de trabajo.

No fusionar archivos de FK si la rama integrada aún no contiene todas sus tablas destino. Las migraciones ya ejecutadas no se renombran ni se reescriben para acomodar la convención: reservar nuevos nombres posteriores. La numeración anterior aplica a la instalación inicial.

Coordinaciones obligatorias entre responsables

Persona 1 con persona 2: periodo, estudiante, sección, solicitud de prematrícula, cupos y origen oficial de notas. Persona 1 con persona 3: empresa, contacto, estudiante, docente, archivos y secciones. Persona 1 con persona 4: curso_plan, plan_estudios, docente, cuentas, archivos y permisos.

Persona 2 con persona 3: definición y momento del egreso a partir de la matrícula e historial; no añadir una FK inexistente sin validar el modelo. Persona 4 con las otras tres: vistas de indicadores para Calidad, con contrato de columnas y permisos. La persona que posee la tabla fuente publica su vista mediante una migración revisada con la persona 4.

Cada persona puede crear modelos de lectura locales para tablas compartidas si sus sistemas son proyectos Laravel separados. Eso no crea tablas nuevas. Mantener esos modelos sincronizados mediante un paquete común o un contrato versionado. No reutilizar contraseñas de base de datos ni asumir que compartir BD inicia una sesión web entre sistemas: el login único requiere diseñar además la autenticación y la sesión o tokens comunes.

Calendario relativo de entregas

Entrega 0 conjunta: diccionario aprobado, versiones fijadas y proyecto central conectado. Entrega 1 paralela: quince esquemas y tablas completas, sin FK, con la distribución 53, 50, 61 y 56. Entrega 2 paralela: relaciones, modelos y restricciones. Entrega 3 paralela: catálogos, pruebas por bloque y vistas. Entrega 4 conjunta: reconstrucción desde cero, flujo entre módulos y ensayo de reversión.

No asignar días cerrados sin conocer dedicación y experiencia. Las entregas son hitos de aceptación: cada persona avanza en los trabajos que no dependen del siguiente hito conjunto. Revisar la carga después de la entrega 1; si se redistribuye trabajo, transferir un esquema completo con responsable único.

Criterios de aceptación por persona

Persona 1: 53 tablas, migración de los quince esquemas, entidades compartidas sin duplicados, login adaptado y coordinación del orden global. Persona 2: 50 tablas, notas y asistencia coherentes, expedientes restringidos y derivaciones verificadas. Persona 3: 61 tablas, catálogos completos, encuestas sin duplicados y referencias a empresas y estudiantes. Persona 4: 56 tablas, versiones académicas trazables, publicación controlada y contratos de vistas de Calidad.

La cantidad de tablas no basta. Cada entrega debe incluir up y down, relaciones Eloquent con claves explícitas, tratamiento de claves compuestas, seeders repetibles y pruebas en PostgreSQL. Revisar permisos según sistema: compartir una base no significa que todos los sistemas puedan leer expedientes sensibles o modificar notas.

Integración y despliegue central

El responsable central ejecuta las migraciones una sola vez por despliegue. Antes de producción: respaldo verificado, restauración ensayada, revisión del SQL y prueba en un entorno equivalente. Desplegar con el usuario propietario y ejecutar php artisan migrate --force. El usuario de ejecución recibe solo los permisos de datos que necesita.

Para auditoria, restringir UPDATE y DELETE de la cuenta de ejecución y revisar los permisos de los triggers si se utilizan. Una cuenta propietaria sigue pudiendo modificar los objetos; también son necesarios controles operativos. Nunca usar migrate:fresh sobre la base compartida para resolver el conflicto de una rama.

18 Errores habituales y solución

relation does not exist: comprobar conexión, esquema, nombre completo y orden de migración. No cambiar search_path para ocultar una referencia equivocada.

column id does not exist: declarar $primaryKey y los nombres de clave en las relaciones. foreign key violation: crear o cargar primero el registro padre y revisar destino y valor; no desactivar FK como solución.

column created_at does not exist: usar $timestamps = false o mapear las columnas temporales de esa tabla. permission denied for schema: revisar USAGE y los privilegios del objeto con el usuario que ejecuta la conexión.

Índice o restricción duplicada: revisar nombres truncados y asignar nombres cortos explícitos. Could not find driver: habilitar pdo_pgsql para el PHP de la consola. Una tabla puente falla al actualizar con save: usar relación N:M, Query Builder con todas las claves o PK técnica aprobada.

19 Lista de cierre

Confirmar los quince esquemas y las 220 tablas por nombre. Comparar columnas y claves con el documento original; completar el diccionario con tipos, nullable, defaults, checks, índices, FK y reglas de borrado. Registrar toda diferencia como decisión de diseño.

Confirmar que persona, usuario, curso, estudiante y empresa no estén duplicados en módulos. Revisar cada relación marcada en el PDF y las dos relaciones pendientes identificadas en este informe. Verificar tablas puente, fechas personalizadas, autenticación y permisos.

Ejecutar migraciones y seeders desde cero en PostgreSQL, pruebas de integridad y de acceso, y una reversión en una base desechable. El cumplimiento de estos puntos habilita la integración; el inventario por sí solo no certifica una base lista para producción.

20 Fuentes y alcance de la verificación

Documento base: esquemas_y_origen_de_tablas.pdf, 14 páginas, entregado por el usuario. Se verificó el inventario de 15 esquemas y 220 tablas. Los tipos y reglas adicionales de los ejemplos se identifican como propuestas.

Documentación oficial consultada el 28 de octubre de 2026:

https://laravel.com/docs/13.x/installation

https://laravel.com/docs/13.x/database

https://laravel.com/docs/13.x/migrations

https://laravel.com/docs/13.x/eloquent

https://laravel.com/docs/13.x/eloquent-relationships

https://laravel.com/docs/13.x/authentication

https://www.postgresql.org/docs/18/ddl-schemas.html

Los ejemplos son instrucciones para implementar; no se ejecutaron migraciones reales de este sistema porque el archivo recibido no contiene un proyecto Laravel ni un diccionario físico completo. La revisión realizada comprende consistencia del inventario, revisión técnica de ejemplos y legibilidad del informe.

Anexo Inventario de implementación de las 220 tablas

Transcripción de nombres del documento fuente, agrupada por esquema. Usar cada entrada como control de creación, modelo o estrategia de acceso, relaciones y prueba. La secuencia de esta lista reproduce el inventario; las fases de migración se definen en la sección 7.

core · 22 tablas

01 pais	02 ubigeo_departamento
03 ubigeo_provincia	04 ubigeo_distrito
05 persona	06 facultad
07 escuela_profesional	08 departamento_academico
09 periodo_academico	10 modalidad_ingreso
11 plan_estudios	12 area_formacion
13 curso	14 curso_plan
15 curso_prerequisito	16 equivalencia_curso
17 docente	18 estudiante
19 ambiente	20 linea_investigacion
21 docente_linea	22 seccion

seguridad · 10 tablas

01 modulo_sistema	02 usuario
03 rol	04 permiso
05 rol_permiso	06 usuario_rol
07 solicitud_acceso	08 token_usuario
09 registro_acceso	10 auditoria

comun · 7 tablas

01 archivo	02 notificacion
03 sector_economico	04 empresa
05 contacto_empresa	06 parametro_sistema
07 dia_no_laborable	

prematricula · 3 tablas

01 cronograma_prematricula	02 solicitud_prematricula
03 detalle_solicitud	

bienestar · 18 tablas

01 especialidad	02 especialista
03 tipo_caso	04 tipo_condicion
05 deporte	06 beneficio
07 perfil_socioeconomico	08 estudiante_deporte
09 estudiante_condicion	10 solicitud_apoyo
11 solicitud_documento	12 caso_bienestar
13 historial_estado_caso	14 horario_disponible
15 cita	16 derivacion
17 seguimiento_derivacion	18 reporte

investigacion_formativa · 11 tablas

01 tipo_producto	02 grupo
03 grupo_estudiante	04 proyecto
05 rubrica	06 criterio
07 entregable	08 entrega
09 evaluacion	10 evaluacion_criterio
11 observacion	

seguimiento_estudiante · 23 tablas

01 perfil_ingreso	02 dimension_brecha
03 brecha_ingreso	04 situacion_academica
05 parametro_sat	06 alerta_sat
07 tutor_asignacion	08 tipo_atencion
09 motivo_atencion	10 solicitud_estudiante
11 atencion	12 atencion_motivo
13 atencion_compromiso	14 atencion_derivacion
15 atencion_evidencia	16 programa_nivelacion
17 nivelacion_grupo	18 nivelacion_participante
19 nivelacion_sesion	20 nivelacion_asistencia
21 nivelacion_criterio_evaluacion	22 nivelacion_calificacion
23 nivelacion_material	

web_institucional · 13 tablas

01 categoria_publicacion	02 publicacion
03 publicacion_archivo	04 campana_admision
05 campana_modalidad	06 cronograma_admision
07 documento_oficial	08 plan_estudios_publicacion
09 perfil_docente_web	10 especialidad
11 docente_especialidad	12 consulta_contacto
13 registro_visita	

calidad · 12 tablas

01 proceso	02 indicador
03 meta_indicador	04 medicion_indicador
05 modelo_acreditacion	06 dimension_acreditacion
07 estandar	08 indicador_estandar
09 evidencia	10 plan_mejora
11 accion_mejora	12 comite_calidad_miembro

gestion_horarios · 11 tablas

01 oferta_curso	02 actividad_academica
03 asignacion_docente	04 bloque_horario
05 disponibilidad_docente	06 ambiente_bloqueo
07 horario	08 detalle_horario
09 restriccion	10 detalle_restriccion
11 horario_revision	

cotecu · 9 tablas

01 miembro_comite	02 lista_cotejo
03 item_lista_cotejo	04 silabo
05 revision_silabo	06 revision_silabo_item
07 revision_curricular	08 revision_curricular_item
09 informe_tecnico	

gestion_practicas · 10 tablas

01 convocatoria	02 observacion_convocatoria
03 practica	04 postulacion
05 seguimiento	06 documento_practica
07 observacion_practica	08 criterio_evaluacion
09 evaluacion	10 evaluacion_criterio

tesis · 22 tablas

01 proyecto_tesis	02 proyecto_estudiante
03 tipo_solicitud	04 solicitud
05 resolucion	06 asesor_proyecto
07 jurado_proyecto	08 colaborador
09 actividad_cronograma	10 recurso
11 partida_presupuesto	12 entrega
13 reporte_similitud	14 rubrica
15 criterio_rubrica	16 revision
17 detalle_evaluacion	18 observacion
19 historial_estado	20 sustentacion
21 resultado_jurado	22 acta

gestion_matriculas · 9 tablas

01 cronograma_matricula	02 limite_creditos
03 matricula	04 matricula_detalle
05 componente_evaluacion	06 nota
07 asistencia	08 acta_notas
09 reserva_matricula	

seguimiento_egresado · 40 tablas

01 cat_area_profesional	02 cat_categoria_habilidad
03 cat_habilidad	04 cat_estado_academico
05 cat_estado_formacion	06 cat_tipo_formacion
07 cat_modalidad_laboral	08 cat_modalidad_trabajo
09 cat_situacion_laboral	10 cat_nivel_logro
11 cat_nivel_satisfaccion	12 cat_canal_envio
13 cohorte	14 competencia
15 objetivo_educacional	16 egresado
17 grado_egresado	18 historial_estado_academico
19 contacto_egresado	20 formacion_posterior
21 importacion_padron	22 importacion_fila
23 campania_seguimiento	24 campania_cohorte
25 campania_canal	26 invitacion_egresado
27 medicion_laboral	28 medicion_habilidad
29 evaluacion_competencia	30 evaluacion_oe
31 satisfaccion_egresado	32 invitacion_empresa
33 respuesta_empresa	34 respuesta_empresa_habilidad
35 indicador_egresado	36 reporte_generado
37 reporte_indicador	38 mejora_curricular
39 plan_accion	40 plan_accion_evidencia